🗺️ SLAM-Toolbox Lifelong Incremental Mapping (Lifelong Mapping)

1. Course Content

- Master the core concepts and engineering value of SLAM-Toolbox lifelong mapping (Lifelong / Continuing Mapping).
- Master the workflow of loading existing historical pose graph (`.posegraph` + `.data`), performing initial relocalization in RViz, and performing incremental mapping in unknown/changed areas.
- Master the pose graph smooth docking mechanism based on lidar point cloud gating (Scan Gating).
- Master the method for serializing and overwriting saving the latest incremental pose graph and 2D occupancy grid map.

---

2. SLAM-Toolbox Lifelong Mapping Introduction

In actual robot applications, the operating environment is often not static (for example, warehouse shelf rearrangements, office area expansions, or newly opened operating passages). Traditional 2D SLAM algorithms (such as Gmapping, Cartographer default configuration) mostly belong to **Single-Session Mapping**; once mapping is completed and a static map is generated, it cannot be directly expanded, and when the environment changes, it often requires completely re-scanning the entire site for mapping, which is extremely costly.

SLAM-Toolbox provides industry-leading **Lifelong / Continuing Mapping** capability:

- **Pose Graph Deserialization**: At system startup, historical topology graph and original laser scan data are directly loaded through the deserialization service, completely restoring the historical environment in the Ceres graph optimization back-end.
- **Initial Pose Smooth Docking**: Through the point cloud gating interception mechanism, laser data is released only after the robot completes initial alignment via RViz calibration or dock parking, avoiding pollution of the historical map by misaligned laser point clouds.
- **Incremental Expansion and Global Loop Closure**: When the robot explores unknown areas or changed environments, it real-time constructs new pose nodes and local scan buffers; when returning to known areas, it automatically triggers global loop closure detection and relaxation optimization between old and new nodes, generating a higher-precision globally unified map.

---

3. Preparation and Running

This example requires opening 3~4 terminals to start the following nodes in sequence:

3.1 Start Lifelong Incremental Mapping Node

Run in **Terminal 1** (specify loading previously saved `map.posegraph` and `map.data` through `pose_graph:=map` parameter):

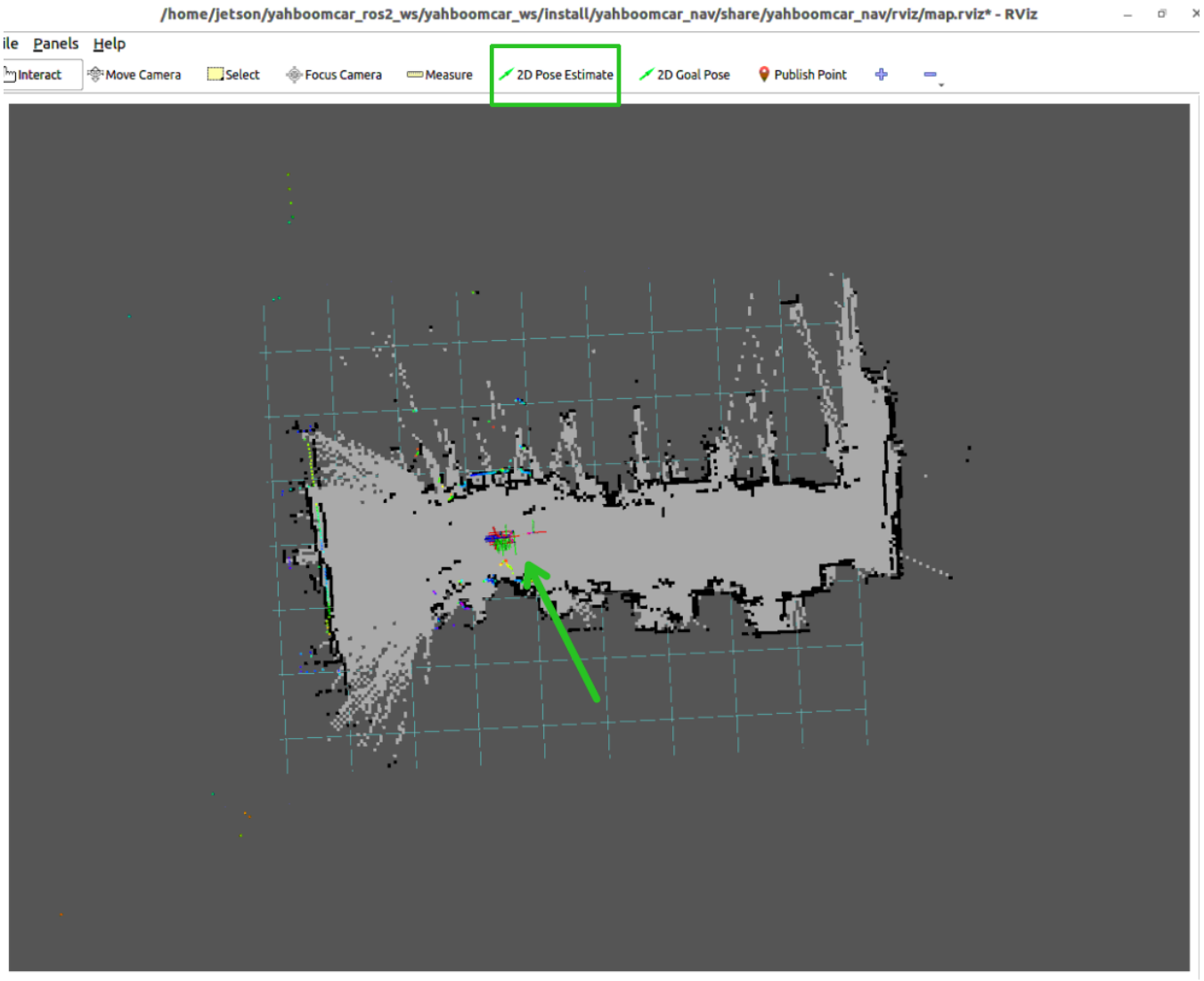
```
ros2 launch yahboomcar_nav map_slam_toolbox_launch.py pose_graph:=map
```

```
 publishing transform
[static_transform_publisher-9] translation: ('0.036000', '0.002000', '0.124000')
[static_transform_publisher-9] rotation: ('0.000000', '0.000000', '0.000000', '1.000000')
[static_transform_publisher-9] from 'base_link' to 'camera_link'
[sync_slam_toolbox_node-11] [INFO] [1788183894.553951073] [slam_toolbox]: Using solver plugin solver_plugins::CeresSolver
[sync_slam_toolbox_node-11] [INFO] [1788183894.558337419] [slam_toolbox]: CeresSolver: Using SCHUR_JACOBI preconditioner.
[sync_slam_toolbox_node-11] Load From File /home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/install/yahboomcar_nav/share/yahboomcar_nav
/maps/map.posegraph
[sync_slam_toolbox_node-11] Mapper <- Module
[sync_slam_toolbox_node-11] Mapper <- m_pSequentialScanMatcher
[sync_slam_toolbox_node-11] Mapper <- m_pGraph
[sync_slam_toolbox_node-11] MapperGraph <- Graph; Graph <- m_Edges; Graph <- m_Vertices
[sync_slam_toolbox_node-11] MapperGraph <- m_pMapper; MapperGraph <- m_pLoopScanMatcher; MapperGraph <- m_pTraversal
[sync_slam_toolbox_node-11] Mapper <- m_pMapperSensorManager
[sync_slam_toolbox_node-11] MapperSensorManager <- m_ScanManagers; MapperSensorManager <- m_Scans
[sync_slam_toolbox_node-11] Mapper <- m_Listeners
[sync_slam_toolbox_node-11] **Finished serializing Mapper**
[sync_slam_toolbox_node-11] Load From File
[sync_slam_toolbox_node-11] **Serializing Dataset**
[sync_slam_toolbox_node-11] Dataset <- m_SensorNameLookup
[sync_slam_toolbox_node-11] Dataset <- m_Data
[sync_slam_toolbox_node-11] Dataset <- m_Lasers
[sync_slam_toolbox_node-11] Dataset <- m_pDatasetInfo
[sync_slam_toolbox_node-11] **Finished serializing Dataset**
[sync_slam_toolbox_node-11] Registering sensor: [Custom Described Lidar]
[sync_slam_toolbox_node-11] WARNING: Logging before InitGoogleLogging() is written to STDERR
[sync_slam_toolbox_node-11] W0831 21:44:54.597888 351895 preprocessor.cc:62] Specified options.num_threads: 50 exceeds maximum avail
able from the threading model Ceres was compiled with: 6.  Bounding to maximum number available.
[ydlidar_ros2_driver_node-7] [YDLIDAR] Lidar running correctly! The health status: bad
[ydlidar_ros2_driver_node-7] [YDLIDAR] Error, YDLidar internal error[0x202] detected. Please reboot the device to retry.
[ydlidar_ros2_driver_node-7] [YDLIDAR] Baseplate device info
[ydlidar_ros2_driver_node-7] Firmware version: 1.0
[ydlidar_ros2_driver_node-7] Hardware version: 2
[ydlidar_ros2_driver_node-7] Model: Tmini Pro
[ydlidar_ros2_driver_node-7] Serial: 2023070500096032
[ydlidar_ros2_driver_node-7] [YDLIDAR] Current scan frequency: 10.00Hz
[ydlidar_ros2_driver_node-7] [YDLIDAR] Lidar init success, Elapsed time 757 ms
[joint_state_publisher-1] [INFO] [1788183895.214208888] [joint_state_publisher]: Waiting for robot_description to be published on th
e robot_description topic...
[imu_attitude_converter-5] [INFO] [1788183895.363590242] [imu_display_tf]: IMU attitude converter: /imu/data_raw -> /imu/data, publi
sh_tf=False
[slam_pose_initializer-12] [INFO] [1788183895.401470181] [slam_pose_initializer]: Waiting for RViz '2D Pose Estimate' on /initialpos
e; laser scans are blocked
[Ackman_driver_A1P-3] [INFO] [1788183895.954839429] [driver_node]: A1 driver initialized: imu_link=imu_link, pub_hz=20.0, arm_status
_hz=5.0, cmd_vel_timeout=0.50s
[ydlidar_ros2_driver_node-7] [YDLIDAR] Create thread 0x63FFE8E0
[ydlidar_ros2_driver_node-7] [YDLIDAR] Successed to start scan mode, Elapsed time 2064 ms
[ydlidar_ros2_driver_node-7] [YDLIDAR] Fixed Size: 404
[ydlidar_ros2_driver_node-7] [YDLIDAR] Sample Rate: 4.00K
[ydlidar_ros2_driver_node-7] [YDLIDAR] Successed to check the lidar, Elapsed time 0 ms
[ydlidar_ros2_driver_node-7] [2026-08-31 21:44:56][info] [YDLIDAR] Now lidar is scanning...
```

3.2 Start RViz Map Visualization and Initial Pose Alignment

Run in **Terminal 2**:

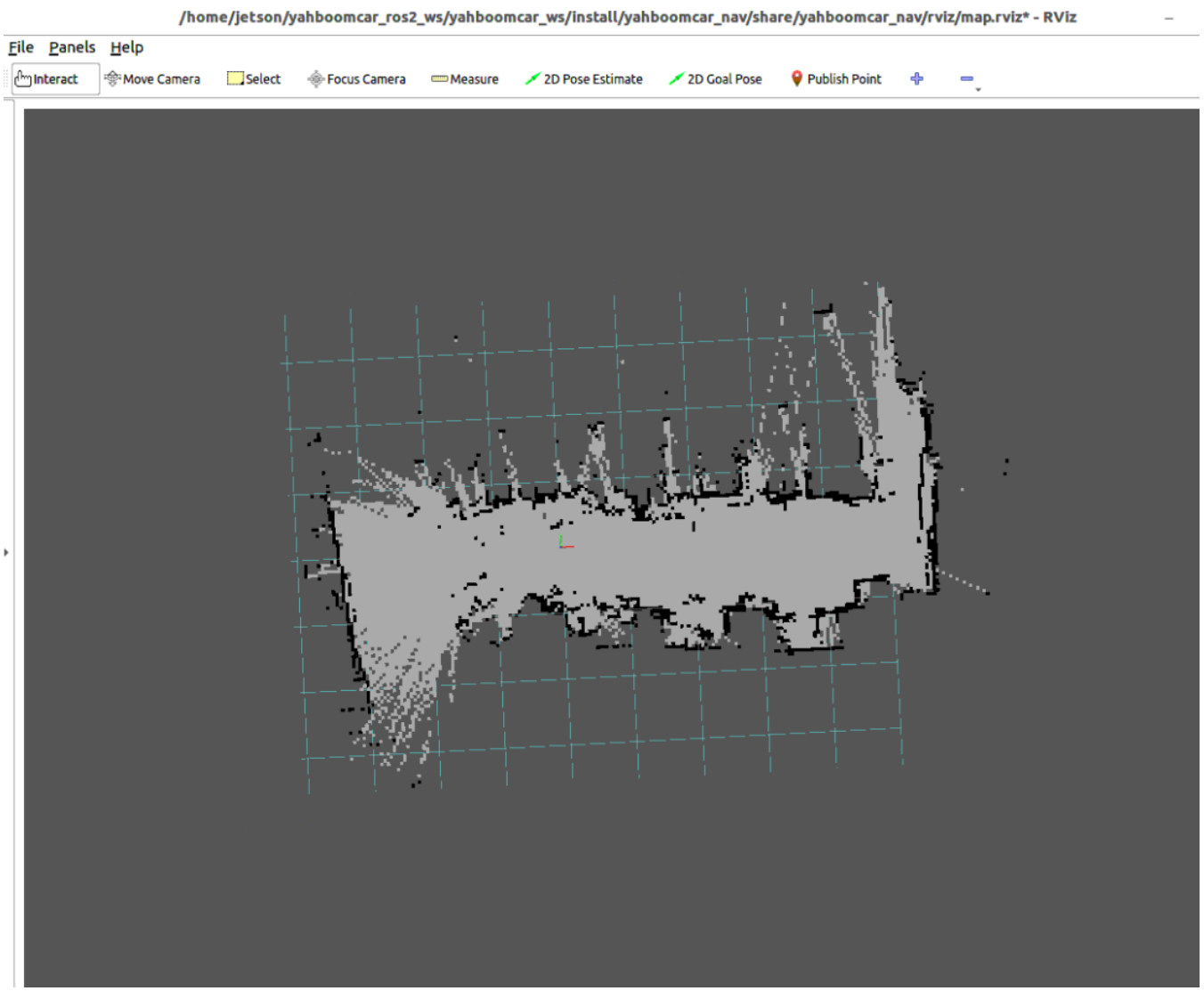
```
ros2 launch yahboomcar_nav display_map_launch.py
```



Important

Initial Relocalization Calibration (if the robot is not stopped at the historical map starting origin):

1. Click the `2D Pose Estimate` tool in the top toolbar of RViz2;
2. Click at the actual robot stopping position on the map, and hold and drag the mouse along the chassis heading direction to release the green arrow to publish `/initialpose`;
3. After the underlying `slam_pose_initializer` node receives the pose, it will automatically call `/slam_toolbox/deserialize_map` service to align the historical map with the current position, and release the lidar point cloud gating to start incremental mapping.



3.3 Start Keyboard Control to Explore New Areas

Run in **Terminal 3** the keyboard control node:

```
ros2 run yahboomcar_ctrl yahboom_keyboard
```

```
jetson@yahboom:~$ ros2 run yahboomcar_ctrl yahboom_keyboard

q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%
t/T : x and y speed switch
s/S : stop keyboard control
space key, k : force stop

Arm Control (publish to 'arm_joint'):
1 - 6 : Select Joint 1~6
a / d : Increase (+) / Decrease (-) selected joint angle (step: 5 deg)
+ / - : Increase (+) / Decrease (-) selected joint angle (step: 5 deg)
0     : Reset all joints to initial pose

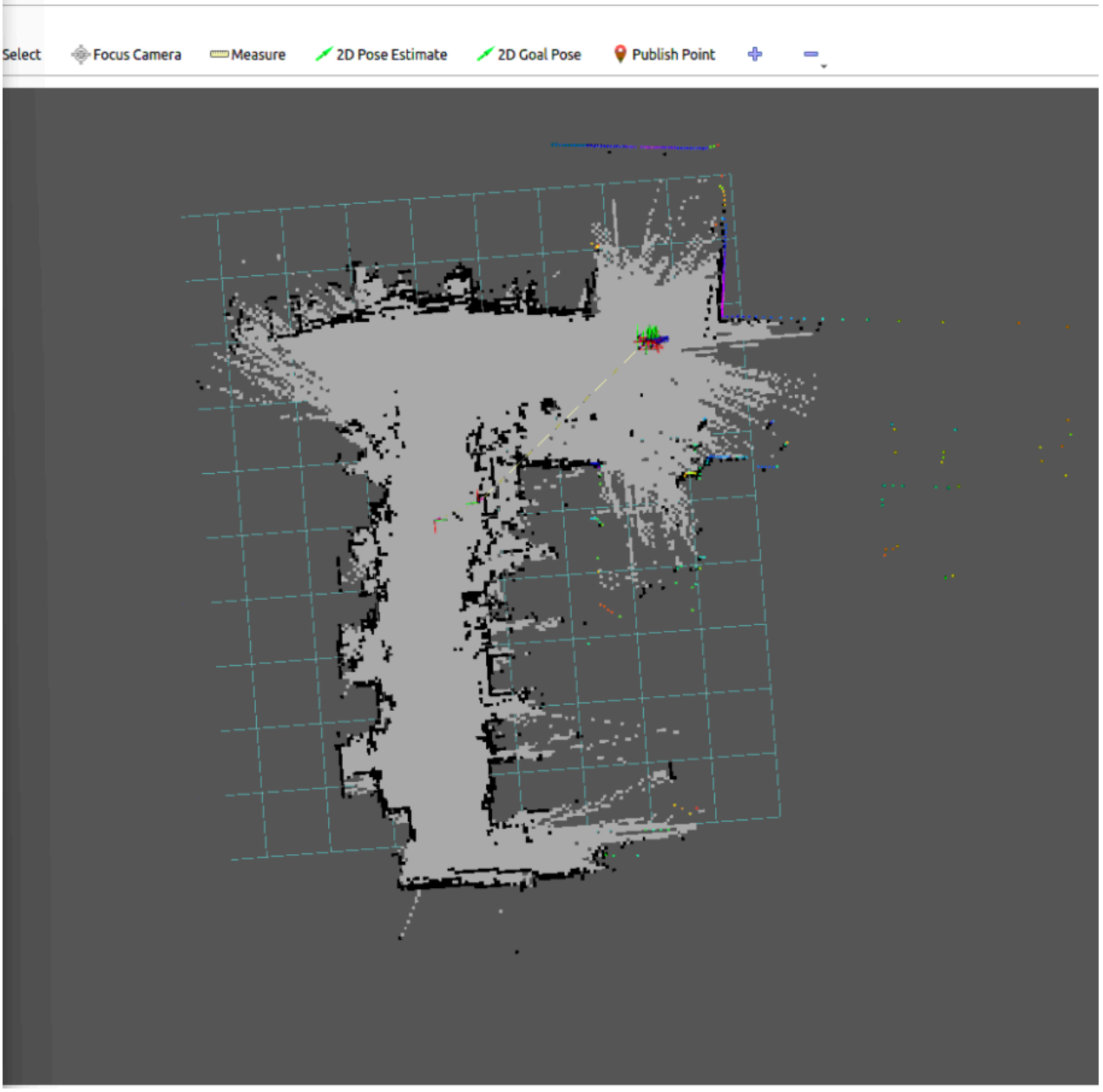
Angle Limits:
Joint 1~4 : 0° ~ 180°
Joint 5   : 0° ~ 270°
Joint 6   : 0° ~ 180°

anything else : stop smoothly
CTRL-C to quit

currently:      speed 0.2       turn 1.0
Current Joint Selection: Joint 1 (90°)
currently:      speed 0.18000000000000002       turn 0.9
currently:      speed 0.16200000000000003       turn 0.81
currently:      speed 0.14580000000000004       turn 0.7290000000000001
currently:      speed 0.13122000000000003       turn 0.6561000000000001
currently:      speed 0.11809800000000004       turn 0.5904900000000002
currently:      speed 0.10628820000000004       turn 0.5314410000000002
currently:      speed 0.09565938000000004       turn 0.47829690000000014
```

3.4 Incremental Mapping Operation and Map Save Instructions

1. **Explore New Environment and Loop Closure Optimization**:
    - Remote control the robot to slowly and smoothly drive from known areas into unknown expansion areas, observe the seamless splicing of historical map boundaries and new map in RViz.
    - After exploring new areas, guide the robot back to known areas; the system will automatically match historical features and trigger global loop closure optimization, eliminating long-distance accumulated drift.



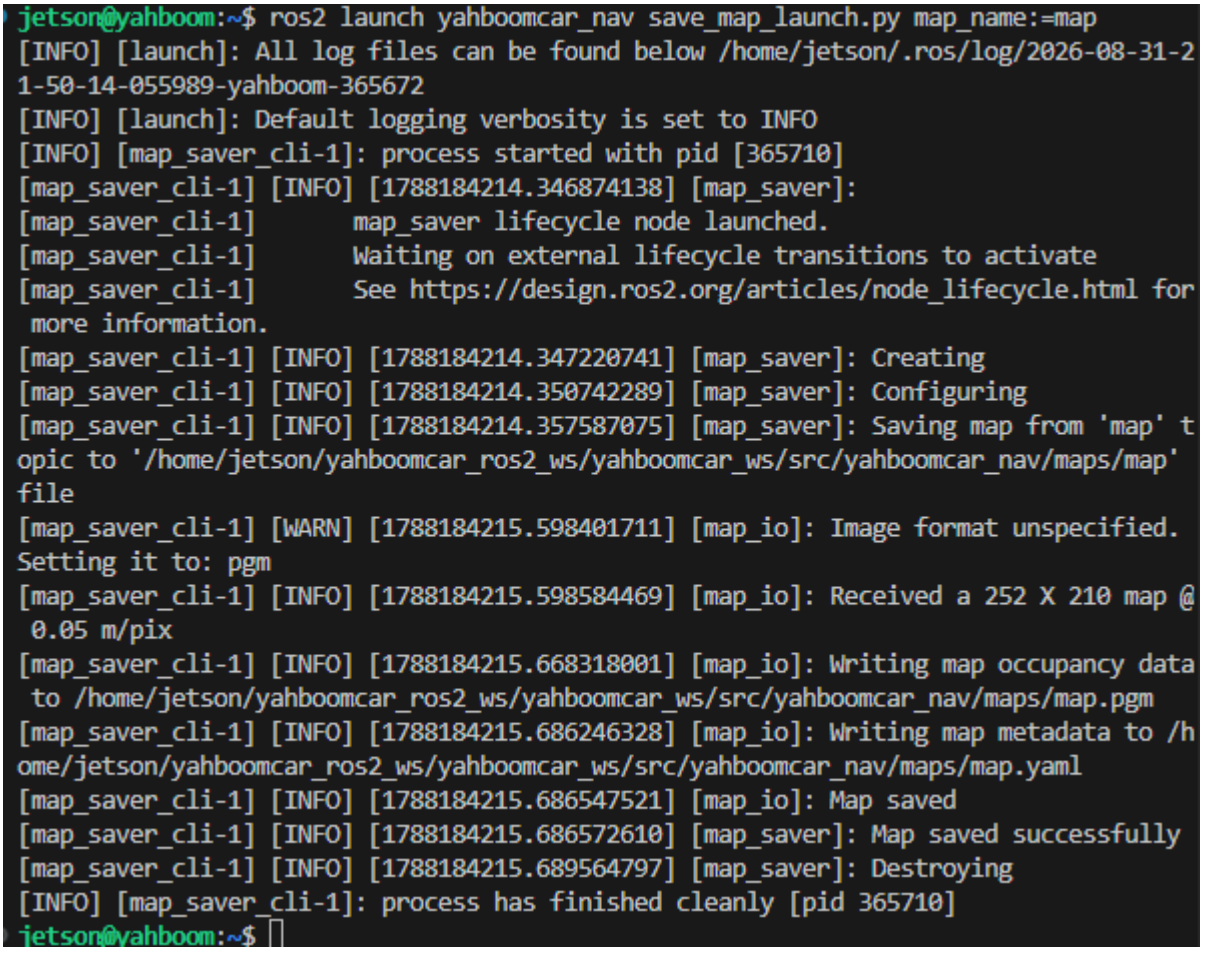
2. **Save Latest Incremental Pose Graph (.posegraph + .data)**:
   Run the pose graph save command in **Terminal 4** to overwrite and update the pose graph file:

```
ros2 launch yahboomcar_nav save_map_launch.py map_type:=posegraph map_name:=map
```

```
jetson@yahboom:~$ ros2 launch yahboomcar_nav save_map_launch.py map_type:=posegraph
map_name:=map
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-08-31-2
1-49-55-503752-yahboom-364872
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [ros2-1]: process started with pid [364913]
[ros2-1] requester: making request: slam_toolbox.srv.SerializePoseGraph_Request(file
name='/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/map')
[ros2-1]
[ros2-1] response:
[ros2-1] slam_toolbox.srv.SerializePoseGraph_Response(result=0)
[ros2-1]
[INFO] [ros2-1]: process has finished cleanly [pid 364913]
jetson@yahboom:~$
```

3. **Save Latest 2D Raster Map (.yaml + .pgm)**:
   Run the raster map save command to generate the latest raster map for subsequent Nav2 navigation:

```
ros2 launch yahboomcar_nav save_map_launch.py map_name:=map
```



> **Note**
>
> After incremental mapping and saving are complete, press `Ctrl+C` in each terminal to stop the program.

---

4. Core Source Code and Algorithm Implementation Mechanism Analysis

4.1 Lifelong Mapping Launch and Scheduling Launch Analysis

Launch File Paths:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/launch/slam/map_slam_toolbox_launch.py
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/launch/slam/slam_toolbox_wit_launch.py
```

Launch Scheduling Core Logic

In `slam_toolbox_wit_launch.py`, the following key scheduling is performed for incremental mapping mode (incoming `pose_graph`):

```
def _resolve_pose_graph(map_folder_dir, pose_graph):
    # 1. Automatically verify historical pose graph dual files (.posegraph and .data)
existence
    graph_path = os.path.expanduser(pose_graph)
    if not os.path.isabs(graph_path):
        graph_path = os.path.join(map_folder_dir, graph_path)

    missing_files = [
        path for path in (f"{graph_path}.posegraph", f"{graph_path}.data")
        if not os.path.isfile(path)
    ]
    if missing_files:
        raise RuntimeError("Cannot continue mapping because pose graph files are missing: "
+ ", ".join(missing_files))
    return graph_path

def _launch_slam_toolbox(context, map_folder_dir, sync_slam_params_file,
async_slam_params_file):
    ...
    if pose_graph:
        pose_graph_path = _resolve_pose_graph(map_folder_dir, pose_graph)
        runtime_parameters["map_file_name"] = pose_graph_path
        runtime_parameters["map_start_at_dock"] = True
        if initial_pose_mode == "rviz":
            # Redirect SLAM node point cloud input to gated and filtered safe topic
            runtime_parameters["scan_topic"] = "/scan_after_initial_pose"

    actions = [
        Node(
            package="slam_toolbox",
            executable=f"{slam_mode}_slam_toolbox_node",
            parameters=parameters,
            name="slam_toolbox",
            output="screen",
        )
    ]

    # When using RViz interactive relocalization, start dedicated pose initializer and point
cloud gating node
    if pose_graph_path and initial_pose_mode == "rviz":
        actions.append(
            Node(
                package="yahboomcar_nav",
                executable="slam_pose_initializer",
                name="slam_pose_initializer",
                output="screen",
                parameters=[{
                    "pose_graph": pose_graph_path,
```

```
                    "map_frame": "map",
                    "initial_pose_topic": "/initialpose",
                    "scan_input_topic": "/scan",
                    "scan_output_topic": "/scan_after_initial_pose",
                    "deserialize_service": "/slam_toolbox/deserialize_map",
                }],
            )
        )
    return actions
```

4.2 Initial Relocalization and Point Cloud Gating Source Analysis (slam_pose_initializer)

Source Code Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/yahboomcar_nav/slam_pose_initializer.py
```

Point Cloud Gating (Scan Gating) and Service Asynchronous Interaction Source

```
class SlamPoseInitializer(Node):
    def __init__(self):
        super().__init__("slam_pose_initializer")
        ...
        self._scan_enabled = False       # Lock point cloud gating in initial state
        self._request_in_flight = False
        self._pending_pose = None

        # 1. Subscribe to original lidar /scan, do not publish to /scan_after_initial_pose when locked
        self._scan_publisher = self.create_publisher(LaserScan, scan_output_topic, qos_profile_sensor_data)
        self._scan_subscription = self.create_subscription(
            LaserScan, scan_input_topic, self._scan_callback, qos_profile_sensor_data
        )

        # 2. Subscribe to 2D Pose Estimate initial pose topic published by RViz
        self._initial_pose_subscription = self.create_subscription(
            PoseWithCovarianceStamped, initial_pose_topic, self._initial_pose_callback, QoSProfile(depth=1)
        )

        # 3. Create SLAM-Toolbox pose graph deserialization service client
        self._deserialize_client = self.create_client(DeserializePoseGraph, service_name)

    def _scan_callback(self, message):
        # Only release lidar data after relocalization alignment is complete
        if self._scan_enabled:
            self._scan_publisher.publish(message)
```

```
    def _initial_pose_callback(self, message):
        if self._scan_enabled or self._request_in_flight:
            return
        ...
        # Extract 2D pose (x, y, yaw)
        self._pending_pose = (position.x, position.y, yaw)
        self._try_initialize()

    def _try_initialize(self):
        x, y, yaw = self._pending_pose
        request = DeserializePoseGraph.Request()
        request.filename = self._pose_graph
        request.match_type = request.START_AT_GIVEN_POSE
        request.initial_pose.x = x
        request.initial_pose.y = y
        request.initial_pose.theta = yaw

        # Asynchronously call deserialization service
        self._request_in_flight = True
        future = self._deserialize_client.call_async(request)
        future.add_done_callback(self._initialization_finished)

    def _initialization_finished(self, future):
        self._request_in_flight = False
        # After deserialization succeeds: enable point cloud gating, unsubscribe from
initialpose
        self._scan_enabled = True
        self.destroy_subscription(self._initial_pose_subscription)
        self.get_logger().info("Pose graph initialized at RViz pose; laser scans are now
enabled")
```

Design Highlights

- **Preventing Historical Map Pollution**: If the robot does not start at the historical origin, directly sending lidar point cloud to SLAM-Toolbox will cause current point cloud to be incorrectly matched near the origin, causing serious map distortion. Through the point cloud gating mechanism, mapping only begins after pose alignment succeeds.
- **Seamless Smooth Docking**: After deserialization succeeds, point cloud is immediately released; the entire process does not require restarting the node, completely transparent to upper-level control and visualization.

4.3 Lifelong Mapping Algorithm Core Parameter Interpretation

Configuration File Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/params/mapper_params_online_sync.y
aml
```

Core Parameter Description Table

Parameter Name	Current Config Value	Core Role and Physical Meaning in Lifelong Mapping
`solver_plugin`	`CeresSolver`	Google Ceres-based nonlinear least squares pose graph optimization back-end
`ceres_loss_function`	`HuberLoss`	Robust kernel function, suppressing distortion from occasional incorrect constraints at old-new area junctions on historical pose graph
`mode`	`mapping`	Mapping working mode (supports continuous new node insertion and graph expansion)
`resolution`	`0.05`	Occupancy grid map resolution (0.05 meters/pixel)
`min_laser_range` / `max_laser_range`	`0.1 / 10.0`	Laser effective filtered distance for incremental mapping (meters)
`minimum_travel_distance`	`0.1`	Insert new pose graph node when straight-line displacement exceeds 0.1m
`minimum_travel_heading`	`0.01745`	Insert new node when rotation displacement exceeds 1°, ensuring dense construction of turns and new area boundaries
`scan_buffer_size`	`30`	Sliding window scan buffer capacity, supporting local dense submap construction
`do_loop_closing`	`true`	Enable global loop closure detection, achieving constraint construction and graph optimization relaxation between old and new areas
`loop_search_maximum_distance`	`4.0`	Maximum candidate spatial search radius for triggering loop matching (meters)
`loop_match_minimum_chain_size`	`12`	Minimum continuous matching chain length for loop closure, ensuring high confidence of old-new map loop closure
`loop_match_minimum_response_fine`	`0.55`	Loop fine matching score threshold, filtering low-confidence matches

Parameter Name	Current Config Value	Core Role and Physical Meaning in Lifelong Mapping
`distance_variance_penalty`	`0.25`	Penalty for displacement variance in weak-feature new areas, improving utilization of chassis odometry prior
`angle_variance_penalty`	`0.25`	Penalty for angular variance in weak-feature new areas